SENATI

Escuela de Tecnologías de la Información

GUÍA 05

Visualización de datos con Matplotlib y Seaborn

Realiza operaciones con las librerías Matplotlib y Seaborn

Escuela	Tecnologías de la Información
Carrera	Ingeniería de Software con Inteligencia Artificial
Módulo formativo	Módulos y Paquetes para Machine Learning con Python
Módulo ocupacional	Especialista en Diseño y Desarrollo de Sistemas de Información con Machine Learning
Semestre / Semana	IV / Semana 4
Tarea de aprendizaje	Realiza operaciones con las Librerías Matplotlib y Seaborn
Docente	Rembrandt Ubalde Enríquez
Estudiante	_____

Entorno de trabajo: Python + Visual Studio Code

Contenido

Contenido 2
Objetivos de la guía 3
Parte 1. Preparación del entorno en Visual Studio Code 3
Paso 1. Crear la carpeta del proyecto 3
Paso 2. Crear y activar el entorno virtual (venv) 3
Paso 3. Instalar las librerías 4
Paso 4. Seleccionar el intérprete y verificar 4
Paso 5. Cómo ejecutar cada script de esta guía 4
Parte 2. Conceptos clave 5
2.1 Importancia de la visualización de datos 5
2.2 ¿Qué son Matplotlib y Seaborn? 5
2.3 Puntos cartesianos 5
2.4 Histogramas 5
2.5 Diagramas de caja (box plots) 6
2.6 Tipos de gráficos estadísticos 6
Parte 3. Prácticas en Visual Studio Code 7
Práctica 01. Puntos en el plano cartesiano 7
Práctica 02. Gráfico de líneas: funciones 8
Práctica 03. Histograma de notas 9
Práctica 04. Diagrama de caja: comparar turnos 10
Práctica 05. Tipos de gráficos en una figura (subplots) 11
Práctica 06. Gráficos estadísticos con Seaborn 12
Práctica 07. Exportar y guardar imágenes 14
Parte 4. Errores frecuentes y soluciones 15
Parte 5. Ejercicios propuestos 16
Lista de verificación 16

(Si el índice aparece vacío, haz clic derecho sobre él y elige "Actualizar campo".)

Objetivos de la guía

Al terminar esta guía serás capaz de:

- **Definir** qué es un histograma y explicar la importancia de la visualización de datos.
- **Definir** las librerías Matplotlib y Seaborn y diferenciar cuándo usar cada una.
- **Crear** gráficos de puntos cartesianos, líneas, barras, circulares, histogramas y diagramas de caja.
- **Exportar y guardar** gráficos en formatos PNG, JPG, PDF y SVG.

Operación	Dónde se trabaja en esta guía
Define los histogramas y la importancia de la visualización de datos	Parte 2 y Prácticas 03 y 04
Define la librería Matplotlib y Seaborn	Parte 2 (sección 2.2), Prácticas 01 a 06
Crea ejemplos de aplicación	Parte 3 (Prácticas 01 a 07) y ejercicios propuestos

Parte 1. Preparación del entorno en Visual Studio Code

Si ya instalaste Python y VS Code en las guías anteriores, no repitas la instalación: empieza en el **Paso 1** creando una carpeta nueva para esta guía y su propio entorno virtual.

Paso 1. Crear la carpeta del proyecto

1. Crea la carpeta **IA_Guia05_Matplotlib_Seaborn** en tu Escritorio o en Documentos.
2. Abre VS Code y elige **Archivo > Abrir carpeta…** (File > Open Folder). Selecciona la carpeta y pulsa **Seleccionar carpeta**.
3. Si aparece el mensaje "¿Confía en los autores de los archivos?", pulsa **Sí, confío en los autores**.
4. Abre la terminal integrada con **Ctrl + ñ** (o menú **Terminal > Nuevo terminal**). Debe abrirse en la ruta de tu carpeta.

> ⚠ **Atención:** Revisa que la ruta de la terminal termine en el nombre de tu carpeta (por ejemplo `...\IA_Guia05_Matplotlib_Seaborn>`). Si dice solo `C:\Users\tu_usuario>`, la carpeta no está abierta: repite el paso 2.

Paso 2. Crear y activar el entorno virtual (venv)

Terminal de VS Code (PowerShell)

```
python -m venv venv
.\venv\Scripts\Activate.ps1
```

Al activarse, la línea de la terminal empieza con **(venv)**. Si aparece un error rojo de "ejecución de scripts deshabilitada", ejecuta lo siguiente, responde **S** y vuelve a activar:

⌨ Solo si aparece el error de scripts

```
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

⚠ **Atención:** Si al escribir `python` se abre la **Microsoft Store** o sale "Python no se encontró", Python no está instalado o no está en el PATH. Reinstálalo desde python.org marcando **"Add python.exe to PATH"**, desactiva en Windows *Configuración > Aplicaciones > Alias de ejecución de aplicaciones* los alias de python.exe y python3.exe, y reinicia VS Code.

Paso 3. Instalar las librerías

⌨ Terminal de VS Code (con (venv) activo)

```
python -m pip install --upgrade pip
pip install numpy pandas matplotlib seaborn
```

Paso 4. Seleccionar el intérprete y verificar

1. Presiona **Ctrl + Shift + P**, escribe **Python: Select Interpreter** y elige la opción que dice **('venv': venv)**.
2. Comprueba en la barra de estado (abajo a la derecha) que aparezca **Python 3.x ('venv')**.
3. Verifica la instalación con el comando de abajo.

⌨ Verificación

```
python -c "import matplotlib, seaborn; print(matplotlib.__version__,
seaborn.__version__)"
```

▶ Resultado esperado en la terminal

```
3.x.x 0.13.x
```

Paso 5. Cómo ejecutar cada script de esta guía

1. En el panel **Explorador** pulsa el ícono **Nuevo archivo** y escribe el nombre exacto indicado (por ejemplo `01_neurona.py`).
2. Copia el código, guarda con **Ctrl + S**.
3. Ejecuta con el botón ▶ **Run Python File** (arriba a la derecha) o escribiendo en la terminal `python nombre_del_archivo.py`.
4. Compara lo que aparece en la terminal con el recuadro verde **Resultado esperado**.

💡 **Consejo:** Cuando un script usa `plt.show()`, el gráfico se abre en una **ventana aparte**. El programa queda en pausa hasta que **cierres esa ventana**; recién entonces la terminal vuelve a estar disponible.

Parte 2. Conceptos clave

2.1 Importancia de la visualización de datos

La **visualización de datos** es la representación gráfica de la información. Una tabla de 1 000 filas no se entiende de un vistazo, pero un gráfico muestra en segundos tendencias, comparaciones y valores extraños. En Machine Learning se usa en todas las etapas: para **explorar** los datos antes de entrenar, para **detectar errores** y valores atípicos, para **evaluar** el modelo y para **comunicar** resultados a personas que no son técnicas.

> 💡 **Consejo:** Regla práctica: antes de entrenar cualquier modelo, grafica tus datos. Muchos problemas (datos mal cargados, unidades distintas, valores imposibles como una nota de 45) se descubren mirando un histograma o un diagrama de caja.

2.2 ¿Qué son Matplotlib y Seaborn?

	Matplotlib	Seaborn
Definición	Librería base de Python para crear gráficos 2D (creada por John D. Hunter, 2003).	Librería construida **sobre Matplotlib** para gráficos **estadísticos** con mejor estética.
Ventaja	Control total de cada detalle: ejes, colores, textos, tamaños.	Gráficos complejos en una línea; trabaja directamente con DataFrames de Pandas.
Importación	`import matplotlib.pyplot as plt`	`import seaborn as sns`
Cuándo usarla	Gráficos personalizados, figuras con varios subgráficos, exportación.	Explorar datos rápido: distribuciones, comparaciones por grupos, correlaciones.

Partes de un gráfico en Matplotlib

- **Figure (figura):** la ventana o "lienzo" completo. Se crea con `plt.figure()` o `plt.subplots()`.
- **Axes (ejes):** cada gráfico dentro de la figura. Una figura puede tener varios.
- **Título, etiquetas y leyenda:** `set_title`, `set_xlabel`, `set_ylabel`, `legend`.

2.3 Puntos cartesianos

El **plano cartesiano** está formado por dos rectas perpendiculares: el eje horizontal **X** (abscisas) y el eje vertical **Y** (ordenadas), que se cruzan en el **origen (0, 0)**. Cada punto se escribe como un par **(x, y)**. Todos los gráficos de Matplotlib se dibujan sobre este sistema: `plt.plot(x, y)` une los puntos con líneas y `plt.scatter(x, y)` los dibuja sueltos.

2.4 Histogramas

Un **histograma** muestra la **distribución** de una variable numérica: divide los valores en intervalos (llamados *bins*) y dibuja una barra cuya altura es la **frecuencia**, es decir, cuántos datos caen en cada intervalo. Permite ver dónde se concentran los datos, si son simétricos y si hay valores lejanos.

2.5 Diagramas de caja (box plots)

Un **diagrama de caja** resume una variable con cinco números y es ideal para **comparar grupos** y **detectar valores atípicos**:

Elemento	Significado
Línea dentro de la caja	**Mediana (Q2)**: el 50 % de los datos está por debajo.
Borde inferior de la caja	**Primer cuartil (Q1)**: el 25 % está por debajo.
Borde superior de la caja	**Tercer cuartil (Q3)**: el 75 % está por debajo.
Altura de la caja (RIC)	**Rango intercuartílico** = Q3 − Q1. Contiene el 50 % central de los datos.
Bigotes	Llegan hasta el dato más lejano que esté dentro de 1.5 × RIC desde la caja.
Puntos fuera de los bigotes	**Valores atípicos** (outliers): datos inusuales que conviene revisar.

2.6 Tipos de gráficos estadísticos

Gráfico	¿Para qué sirve?	Matplotlib	Seaborn
Líneas	Ver la evolución en el tiempo.	`plt.plot`	`sns.lineplot`
Barras	Comparar cantidades entre categorías.	`plt.bar`	`sns.barplot`
Circular (pie)	Mostrar partes de un total (pocas categorías).	`plt.pie`	—
Dispersión	Ver la relación entre dos variables numéricas.	`plt.scatter`	`sns.scatterplot`
Histograma	Ver la distribución de una variable.	`plt.hist`	`sns.histplot`
Caja (box plot)	Comparar grupos y detectar atípicos.	`plt.boxplot`	`sns.boxplot`
Mapa de calor	Mostrar una matriz de valores, como correlaciones.	`plt.imshow`	`sns.heatmap`

Parte 3. Prácticas en Visual Studio Code

Práctica 01. Puntos en el plano cartesiano

01_puntos_cartesianos.py

```
# 01_puntos_cartesianos.py - Puntos en el plano cartesiano con Matplotlib
import matplotlib
import matplotlib.pyplot as plt

print("Versión de Matplotlib:", matplotlib.__version__)

# Cada punto es un par (x, y)
x = [-3, -1, 0, 2, 4]
y = [2, -2, 1, 3, -1]

fig, ax = plt.subplots(figsize=(6, 5))
ax.scatter(x, y, color="tab:blue", s=80, zorder=3)      # s = tamaño del punto

# Etiquetar cada punto con sus coordenadas
for xi, yi in zip(x, y):
    ax.annotate(f"({xi}, {yi})", (xi, yi), textcoords="offset points", xytext=(6, 6))

# Dibujar los ejes X e Y que pasan por el origen
ax.axhline(0, color="black", linewidth=1)
ax.axvline(0, color="black", linewidth=1)
ax.set_xlim(-5, 5)
ax.set_ylim(-4, 4)
ax.grid(True, linestyle=":")
ax.set_title("Puntos en el plano cartesiano")
ax.set_xlabel("Eje X")
ax.set_ylabel("Eje Y")

plt.savefig("01_puntos_cartesianos.png", dpi=100, bbox_inches="tight")
print("Imagen guardada: 01_puntos_cartesianos.png")
plt.show()
```

▶ Resultado esperado en la terminal

```
Versión de Matplotlib: 3.11.2
Imagen guardada: 01_puntos_cartesianos.png
```

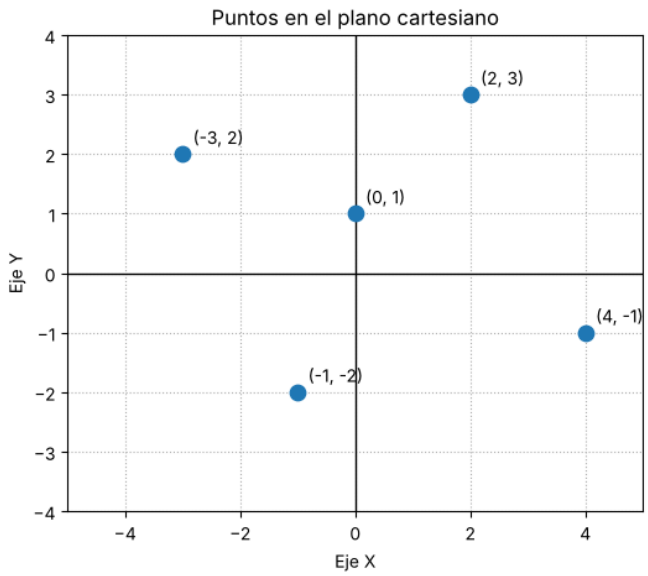


Figura 1. Puntos ubicados en los cuatro cuadrantes del plano

`axhline` y `axvline` dibujan los ejes que pasan por el origen; `annotate` escribe el par (x, y) junto a cada punto.

Práctica 02. Gráfico de líneas: funciones

02_lineas_funciones.py

```
# 02_lineas_funciones.py - Unir puntos para graficar funciones
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(-3, 3, 100)          # 100 valores de x entre -3 y 3
y_lineal = 2 * x + 1                 # recta
y_cuadratica = x ** 2                # parábola

plt.figure(figsize=(7, 4))
plt.plot(x, y_lineal, label="y = 2x + 1", color="tab:blue")
plt.plot(x, y_cuadratica, label="y = x²", color="tab:orange", linestyle="--")
plt.axhline(0, color="gray", linewidth=0.8)
plt.axvline(0, color="gray", linewidth=0.8)
plt.title("Gráfico de líneas: funciones matemáticas")
plt.xlabel("x")
plt.ylabel("y")
plt.legend()
plt.grid(True, alpha=0.3)

plt.savefig("02_lineas_funciones.png", dpi=100, bbox_inches="tight")
print("Imagen guardada: 02_lineas_funciones.png")
plt.show()
```

▶ Resultado esperado en la terminal

```
Imagen guardada: 02_lineas_funciones.png
```

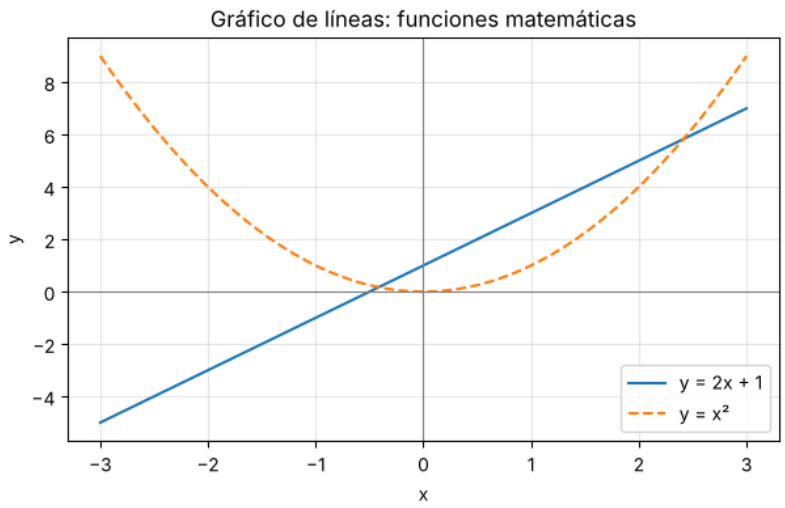


Figura 2. Una recta y una parábola en el mismo gráfico

Práctica 03. Histograma de notas

03_histograma.py

```
# 03_histograma.py - Distribución de notas con un histograma
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(42)               # semilla: siempre los mismos datos
notas = rng.normal(loc=13, scale=2.5, size=200).clip(0, 20).round()

print("Cantidad de notas:", len(notas))
print("Promedio:", round(notas.mean(), 2), "| Mínima:", notas.min(), "| Máxima:",
notas.max())

plt.figure(figsize=(7, 4))
frecuencias, bordes, _ = plt.hist(notas, bins=10, color="tab:green", edgecolor="black")
plt.axvline(notas.mean(), color="red", linestyle="--", label=f"Promedio =
{notas.mean():.1f}")
plt.title("Histograma de notas (200 estudiantes)")
plt.xlabel("Nota")
plt.ylabel("Frecuencia (cantidad de estudiantes)")
plt.legend()

print("Frecuencia por intervalo:", frecuencias.astype(int))
plt.savefig("03_histograma.png", dpi=100, bbox_inches="tight")
print("Imagen guardada: 03_histograma.png")
plt.show()
```

▶ Resultado esperado en la terminal

```
Cantidad de notas: 200
Promedio: 12.94 | Mínima: 8.0 | Máxima: 20.0
Frecuencia por intervalo: [10 17 27 33 33 59  8  9  3  1]
Imagen guardada: 03_histograma.png
```

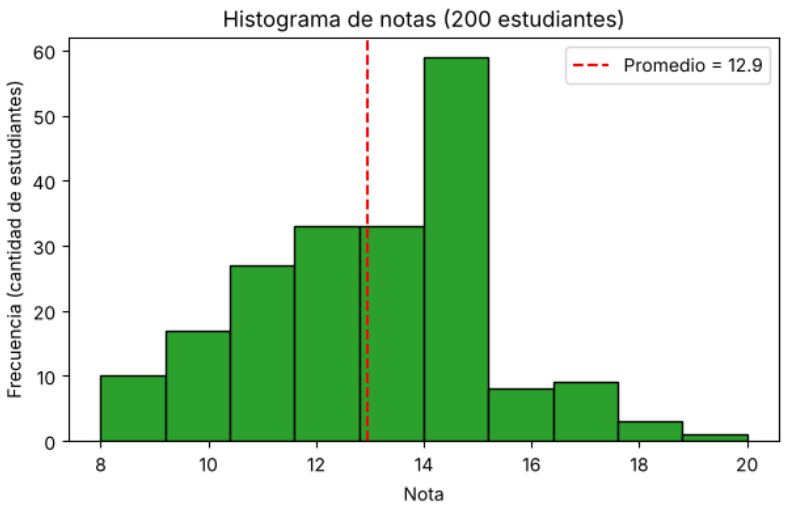


Figura 3. Distribución de 200 notas en 10 intervalos

Interpretación: la mayoría de notas se concentra entre 12 y 15, y pocas superan 17. El parámetro `bins=10` define la cantidad de intervalos; prueba con 5 y con 20 para ver cómo cambia la forma.

Práctica 04. Diagrama de caja: comparar turnos

04_boxplot.py

```
# 04_boxplot.py - Diagrama de caja (box plot) para comparar grupos
import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(7)
turno_manana = rng.normal(14, 2, 40).round()
turno_tarde = rng.normal(12, 3, 40).round()
turno_noche = np.append(rng.normal(13, 1.5, 39).round(), 3)   # incluye un valor atípico

grupos = [turno_manana, turno_tarde, turno_noche]
nombres = ["Mañana", "Tarde", "Noche"]

# Los 5 números que resume una caja
for nombre, g in zip(nombres, grupos):
    q1, med, q3 = np.percentile(g, [25, 50, 75])
    print(f"{nombre:7} -> mín={g.min():.0f}  Q1={q1:.1f}  mediana={med:.1f}  Q3={q3:.1f}
máx={g.max():.0f}  RIC={q3 - q1:.1f}")

plt.figure(figsize=(7, 4))
plt.boxplot(grupos, tick_labels=nombres, patch_artist=True,
            boxprops=dict(facecolor="lightblue"))
plt.title("Notas por turno (diagrama de caja)")
plt.ylabel("Nota")
plt.grid(axis="y", alpha=0.3)

plt.savefig("04_boxplot.png", dpi=100, bbox_inches="tight")
print("Imagen guardada: 04_boxplot.png")
plt.show()
```

▶ Resultado esperado en la terminal

```
Mañana  -> mín=9  Q1=12.0  mediana=13.0  Q3=14.0  máx=17  RIC=2.0
Tarde   -> mín=6  Q1=10.8  mediana=12.0  Q3=14.0  máx=18  RIC=3.2
Noche   -> mín=3  Q1=12.0  mediana=13.0  Q3=14.0  máx=16  RIC=2.0
```

```
Imagen guardada: 04_boxplot.png
```

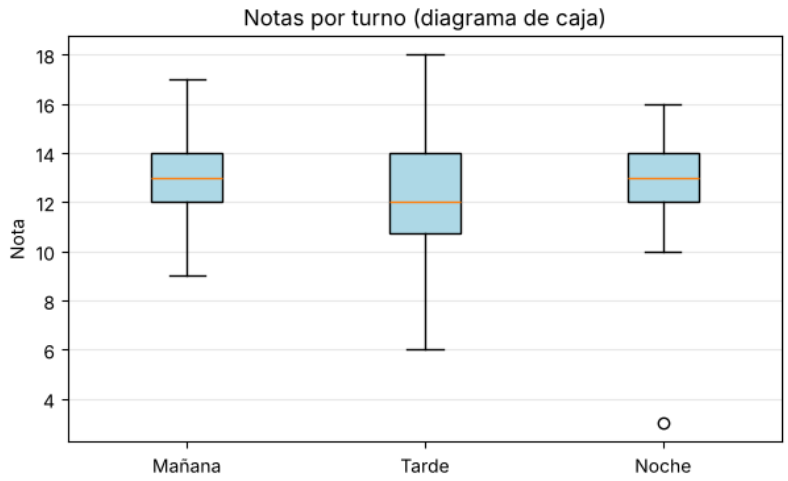


Figura 4. Notas por turno; el círculo inferior del turno noche es un valor atípico

Interpretación: el turno tarde tiene la caja más alta (RIC = 3.2), es decir, sus notas son las más **dispersas**. En el turno noche aparece un punto aislado en la nota 3: es un **valor atípico** que conviene revisar (¿error de digitación o un caso real?).

Práctica 05. Tipos de gráficos en una figura (subplots)

05_tipos_graficos.py

```
# 05_tipos_graficos.py - Varios tipos de gráficos en una sola figura
import matplotlib.pyplot as plt

meses = ["Ene", "Feb", "Mar", "Abr", "May", "Jun"]
ventas = [120, 135, 110, 160, 175, 190]
productos = ["Laptops", "Mouse", "Teclados", "Monitores"]
unidades = [45, 120, 80, 30]
horas = [2, 3, 4, 5, 6, 7, 8, 9]
notas = [9, 10, 12, 12, 14, 15, 17, 18]

fig, ejes = plt.subplots(2, 2, figsize=(10, 7))   # cuadrícula de 2 x 2

ejes[0, 0].plot(meses, ventas, marker="o")
ejes[0, 0].set_title("Líneas: ventas por mes")

ejes[0, 1].bar(productos, unidades, color="tab:orange")
ejes[0, 1].set_title("Barras: unidades vendidas")

ejes[1, 0].pie(unidades, labels=productos, autopct="%1.0f%%")
ejes[1, 0].set_title("Circular (pie): participación")

ejes[1, 1].scatter(horas, notas, color="tab:purple")
ejes[1, 1].set_title("Dispersión: horas de estudio vs nota")
ejes[1, 1].set_xlabel("Horas")
ejes[1, 1].set_ylabel("Nota")

fig.suptitle("Tipos de gráficos estadísticos", fontsize=14)
fig.tight_layout()
plt.savefig("05_tipos_graficos.png", dpi=100, bbox_inches="tight")
print("Imagen guardada: 05_tipos_graficos.png")
```

```
plt.show()
```

▶ Resultado esperado en la terminal

```
Imagen guardada: 05_tipos_graficos.png
```

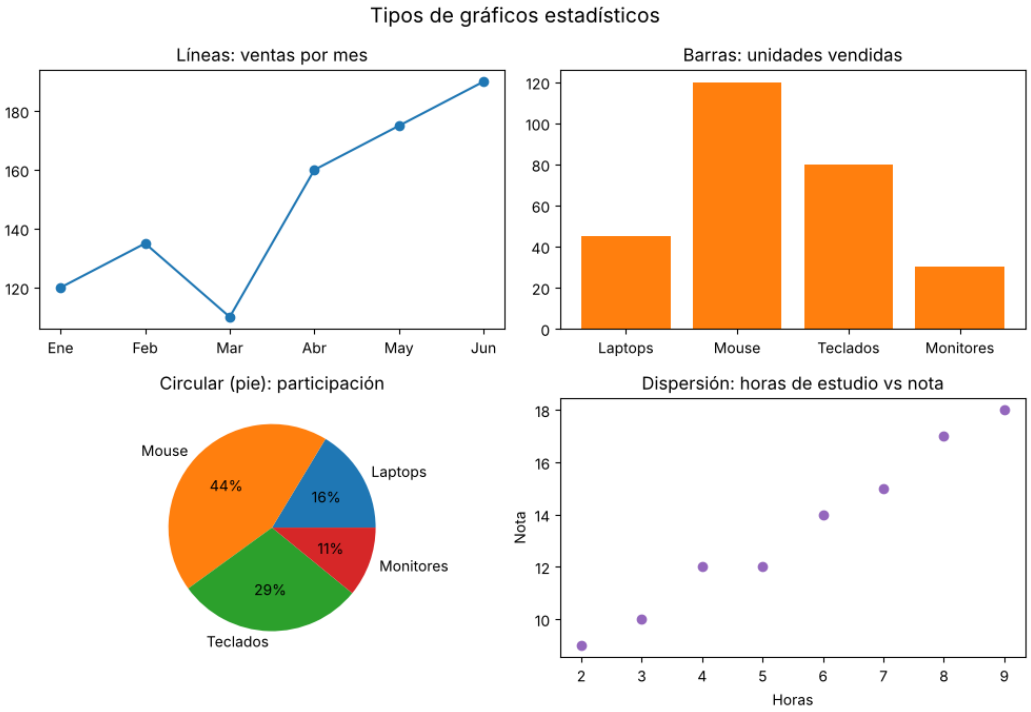


Figura 5. Cuatro tipos de gráficos en una cuadrícula de 2 × 2

> 💡 **Consejo:** `plt.subplots(2, 2)` devuelve una matriz de ejes. Se accede a cada gráfico con `ejes[fila, columna]`, empezando desde 0.

Práctica 06. Gráficos estadísticos con Seaborn

Seaborn trabaja con **DataFrames**: le indicas el nombre de las columnas en x, y y hue (color por grupo) y él se encarga del resto.

06_seaborn.py

```
# 06_seaborn.py - Gráficos estadísticos con Seaborn y un DataFrame
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

print("Versión de Seaborn:", sns.__version__)
sns.set_theme(style="whitegrid")              # estilo general de Seaborn

# Crear un DataFrame de 150 estudiantes
rng = np.random.default_rng(1)
n = 150
carrera = rng.choice(["Software", "Redes", "Datos"], n)
horas = rng.uniform(1, 10, n).round(1)
```

```
nota = (8 + 0.9 * horas + rng.normal(0, 1.5, n)).clip(0, 20).round()
df = pd.DataFrame({"carrera": carrera, "horas_estudio": horas, "nota": nota})
print(df.head())
print(df.groupby("carrera")["nota"].mean().round(2))

fig, ejes = plt.subplots(2, 2, figsize=(11, 8))
sns.histplot(data=df, x="nota", kde=True, ax=ejes[0, 0])
ejes[0, 0].set_title("histplot: distribución de notas")

sns.boxplot(data=df, x="carrera", y="nota", hue="carrera", ax=ejes[0, 1])
ejes[0, 1].set_title("boxplot: notas por carrera")

sns.scatterplot(data=df, x="horas_estudio", y="nota", hue="carrera", ax=ejes[1, 0])
ejes[1, 0].set_title("scatterplot: horas vs nota")

sns.heatmap(df[["horas_estudio", "nota"]].corr(), annot=True, cmap="Blues", ax=ejes[1,
1])
ejes[1, 1].set_title("heatmap: correlación")

fig.tight_layout()
plt.savefig("06_seaborn.png", dpi=100, bbox_inches="tight")
print("Imagen guardada: 06_seaborn.png")
plt.show()
```

▶ Resultado esperado en la terminal

```
Versión de Seaborn: 0.13.2
    carrera  horas_estudio  nota
0     Redes            1.2  10.0
1     Redes            7.1  14.0
2     Datos            9.3  16.0
3     Datos            8.4  15.0
4  Software            9.0  15.0
carrera
Datos       12.74
Redes       12.81
Software    12.73
Name: nota, dtype: float64
Imagen guardada: 06_seaborn.png
```

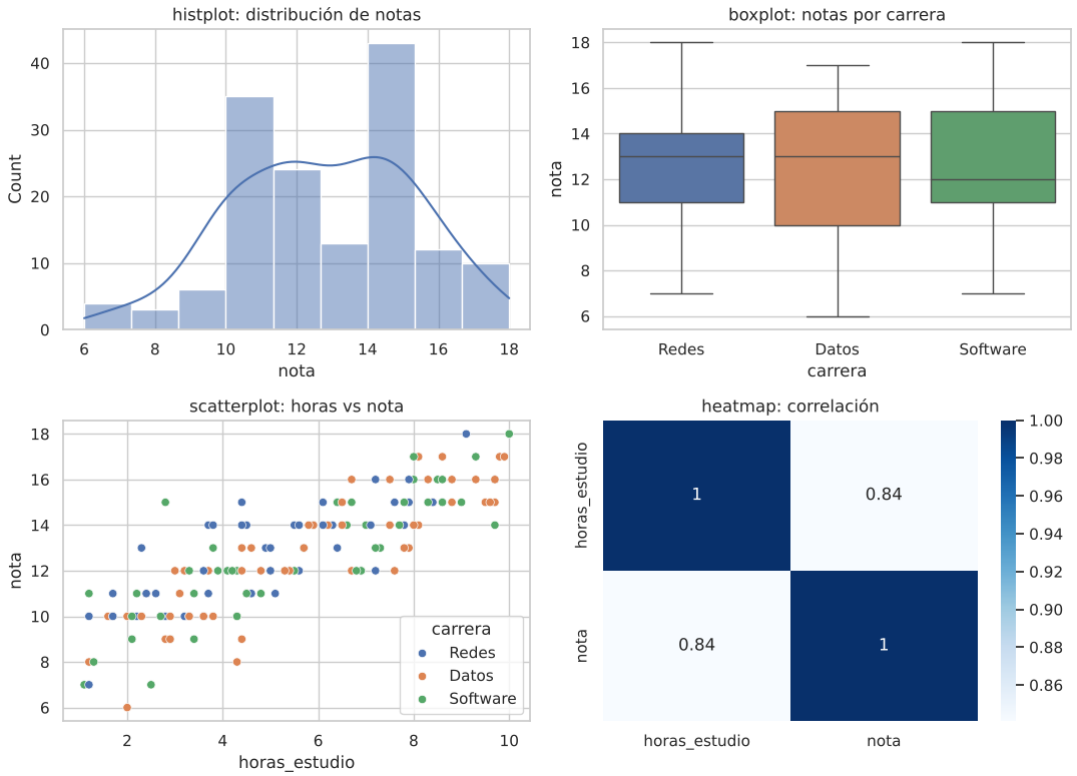


Figura 6. Histograma con curva KDE, box plot por carrera, dispersión y mapa de calor

Interpretación: el gráfico de dispersión muestra que a más horas de estudio, mayor nota; el mapa de calor lo confirma con una **correlación de 0.84** (1 sería una relación perfecta). La opción `kde=True` agrega una curva suave que estima la forma de la distribución.

Práctica 07. Exportar y guardar imágenes

07_exportar.py

```
# 07_exportar.py - Exportar y guardar gráficos en distintos formatos
import os
import matplotlib.pyplot as plt

os.makedirs("graficos", exist_ok=True)       # crea la carpeta si no existe

meses = ["Ene", "Feb", "Mar", "Abr"]
asistencia = [92, 88, 95, 90]

fig, ax = plt.subplots(figsize=(6, 4))
ax.bar(meses, asistencia, color="tab:cyan")
ax.set_title("Asistencia promedio (%)")
ax.set_ylim(0, 100)

# savefig SIEMPRE antes de plt.show()
fig.savefig("graficos/asistencia.png", dpi=150, bbox_inches="tight")   # imagen para
Word/PPT
fig.savefig("graficos/asistencia.jpg", dpi=150, bbox_inches="tight")   # imagen
comprimida
fig.savefig("graficos/asistencia.pdf", bbox_inches="tight")            # vectorial para
imprimir
fig.savefig("graficos/asistencia.svg", bbox_inches="tight")            # vectorial para
web
```

```
fig.savefig("graficos/asistencia_transparente.png", transparent=True)  # fondo
transparente

for archivo in sorted(os.listdir("graficos")):
    kb = os.path.getsize(os.path.join("graficos", archivo)) / 1024
    print(f"{archivo:30} {kb:6.1f} KB")
plt.show()
```

▶ **Resultado esperado en la terminal**

```
asistencia.jpg                  27.6 KB
asistencia.pdf                  10.0 KB
asistencia.png                  19.4 KB
asistencia.svg                  19.8 KB
asistencia_transparente.png      9.1 KB
```

Formato	Tipo	Uso recomendado
PNG	Imagen (píxeles)	Informes en Word, presentaciones. Admite fondo transparente.
JPG	Imagen comprimida	Fotos o envío por correo/WhatsApp; puede perder nitidez.
PDF	Vectorial	Impresión: se ve nítido a cualquier tamaño.
SVG	Vectorial	Páginas web; se puede editar en Inkscape o Illustrator.

> ⚠ **Atención:** Llama a `savefig()` **antes** de `plt.show()`. Al cerrar la ventana de `show()` la figura se borra de memoria y la imagen guardada saldría **en blanco**.

- `dpi=150`: resolución en puntos por pulgada (300 para impresión de alta calidad).
- `bbox_inches="tight"`: recorta los márgenes blancos sobrantes.
- `transparent=True`: fondo transparente, útil para poner el gráfico sobre una diapositiva de color.

Parte 4. Errores frecuentes y soluciones

Error o síntoma	Causa	Solución
`ModuleNotFoundError: No module named 'seaborn'`	No se instaló o el venv no está activo.	Activa el venv y ejecuta `pip install seaborn`.
El script termina y no aparece el gráfico	Falta `plt.show()`.	Agrega `plt.show()` al final.
La terminal se queda “colgada”	La ventana del gráfico sigue abierta.	Cierra la ventana del gráfico.
La imagen guardada sale en blanco	`savefig` se llamó después de `show`.	Coloca `savefig` antes de `plt.show()`.
`ValueError: x and y must have same first dimension`	Las listas x e y tienen distinto tamaño.	Verifica con `len(x)` y `len(y)`.
Los textos se superponen	Figura pequeña o muchos subgráficos.	Usa `figsize` más grande y `fig.tight_layout()`.

Error o síntoma	Causa	Solución
Tildes o ñ con símbolos raros en el título	Archivo guardado con otra codificación.	En VS Code guarda el archivo como **UTF-8** (barra de estado, abajo a la derecha).

Parte 5. Ejercicios propuestos

1. Grafica en el plano cartesiano los vértices de un triángulo A(−2, −1), B(3, −1) y C(1, 4), y únelos con `plt.plot` para cerrar la figura.
2. Grafica en una misma figura $y = x^3$ e $y = -x^2 + 4$ en el intervalo [−3, 3], con leyenda y cuadrícula.
3. Crea un histograma con las edades de 100 personas generadas con `rng.integers(17, 40, 100)`. Prueba `bins=5` y `bins=15` y explica la diferencia.
4. Con un DataFrame de ventas por tienda (3 tiendas, 30 días cada una), dibuja un `sns.boxplot` y responde: ¿qué tienda tiene ventas más estables?
5. Lee el archivo `notas_procesadas.csv` de la Guía 01 con Pandas y crea un gráfico de barras con el promedio de cada alumno.
6. **Reto:** arma un "mini dashboard" de 2 × 2 con Seaborn sobre un tema de tu elección y expórtalo en PNG (dpi=300) y en PDF dentro de la carpeta `graficos`.

Lista de verificación

- ☐ Instalé Matplotlib y Seaborn en el venv y la verificación muestra sus versiones.
- ☐ Puedo explicar por qué es importante visualizar los datos antes de entrenar un modelo.
- ☐ Ubiqué puntos en el plano cartesiano y grafiqué funciones con líneas.
- ☐ Creé e interpreté un histograma.
- ☐ Creé un diagrama de caja e identifiqué mediana, cuartiles y valores atípicos.
- ☐ Combiné varios tipos de gráficos con `subplots`.
- ☐ Usé Seaborn con un DataFrame (histplot, boxplot, scatterplot, heatmap).
- ☐ Exporté gráficos en PNG, JPG, PDF y SVG.
- ☐ Resolví al menos 4 ejercicios propuestos.